

DMI COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
B.E / B.TECH AUTONOMOUS END SEMESTER PRACTICAL EXAMINATION, APR /
MAY-2026

CS1256 - DATABASE MANAGEMENT SYSTEMS LABORATORY

Fourth Semester (Regulation 2024)

SOLVED ANSWER KEY – ALL QUESTIONS

Aim and Procedure / E-R Diagram	SQL Query and Program	Execution and Results	Viva-Voce	Record	Total
20	30	30	10	10	100

Time: 3 Hours (Answer any one Question) Max. Marks: 100

1. Library Database

AIM:

To create a Library database with BOOK, BOOK_AUTHORS and PUBLISHER relations (with supporting BORROWER and BOOK_LENDING tables for lending operations) and to execute join, aggregate, delete and view operations on it.

E-R DIAGRAM:

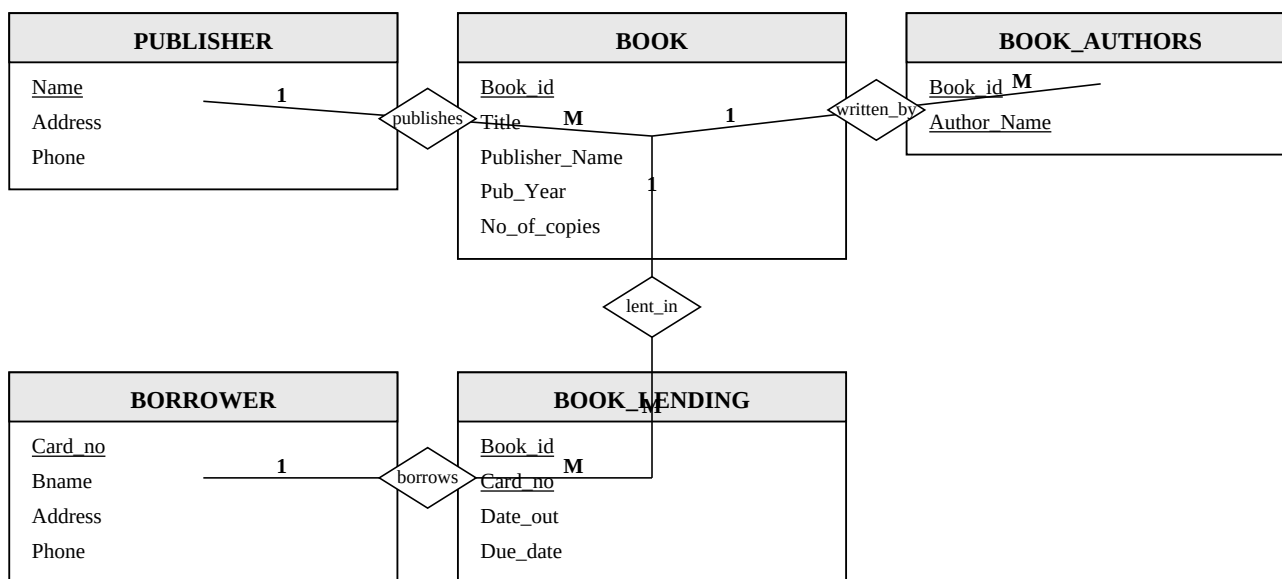


TABLE CREATION:

```

CREATE TABLE PUBLISHER (
    Name VARCHAR(40) PRIMARY KEY,
    Address VARCHAR(60),
    Phone VARCHAR(15));

CREATE TABLE BOOK (
    Book_id INT PRIMARY KEY,
    Title VARCHAR(60),
    Publisher_Name VARCHAR(40) REFERENCES PUBLISHER(Name),
    Pub_Year INT,
    No_of_copies INT);
  
```

```

CREATE TABLE BOOK_AUTHORS (
    Book_id      INT REFERENCES BOOK(Book_id) ON DELETE CASCADE,
    Author_Name  VARCHAR(40),
    PRIMARY KEY (Book_id, Author_Name));

CREATE TABLE BORROWER (
    Card_no      INT PRIMARY KEY,
    Bname        VARCHAR(40),
    Address       VARCHAR(60),
    Phone        VARCHAR(15));

CREATE TABLE BOOK_LENDING (
    Book_id      INT REFERENCES BOOK(Book_id) ON DELETE CASCADE,
    Card_no      INT REFERENCES BORROWER(Card_no),
    Date_out     DATE,
    Due_date     DATE,
    PRIMARY KEY (Book_id, Card_no));

```

SQL QUERIES:

a) Details of all books – id, title, publisher, author:

```

SELECT B.Book_id, B.Title, B.Publisher_Name, A.Author_Name
FROM   BOOK B JOIN BOOK_AUTHORS A ON B.Book_id = A.Book_id
ORDER BY B.Book_id;

```

b) Borrowers who borrowed more than 3 books between Jan–Jun 2024:

```

SELECT BR.Card_no, BR.Bname, COUNT(*) AS Books_Borrowed
FROM   BORROWER BR JOIN BOOK_LENDING L ON BR.Card_no = L.Card_no
WHERE  L.Date_out BETWEEN '2024-01-01' AND '2024-06-30'
GROUP BY BR.Card_no, BR.Bname
HAVING COUNT(*) > 3;

```

c) Delete a book and reflect in other tables:

```

-- ON DELETE CASCADE removes dependent rows automatically.
DELETE FROM BOOK WHERE Book_id = 101;
-- BOOK_AUTHORS and BOOK_LENDING rows for 101 are removed by cascade.

```

d) View of available copies:

```

CREATE VIEW AVAILABLE_BOOKS AS
SELECT Book_id, Title, No_of_copies
FROM   BOOK
WHERE  No_of_copies > 0;

SELECT * FROM AVAILABLE_BOOKS;

```

SAMPLE OUTPUT:

```

a)
BOOK_ID  TITLE                PUBLISHER_NAME  AUTHOR_NAME
-----  -
101      DBMS Concepts       McGraw Hill     Silberschatz
101      DBMS Concepts       McGraw Hill     Korth
102      Operating Systems   Pearson         Galvin

b)
CARD_NO  BNAME    BOOKS_BORROWED
-----  -
12       Anand    4

d)
BOOK_ID  TITLE                NO_OF_COPIES
-----  -
102      Operating Systems   3
103      Computer Networks  2

```

RESULT:

Thus the Library database was created and all the queries were executed successfully and the output verified.

2. EMPLOYEE table – DDL operations and trigger

AIM:

To create an EMPLOYEE table and demonstrate ALTER, table copy, DELETE, DROP operations and a trigger for automatic updation.

TABLE STRUCTURE:

EMPLOYEE
<u>SNo</u>
Name
Designation
Branch

SQL QUERIES:

```
CREATE TABLE employee (  
    SNo          INT PRIMARY KEY,  
    Name         VARCHAR(30),  
    Designation  VARCHAR(30),  
    Branch       VARCHAR(30));
```

a) Add column salary:

```
ALTER TABLE employee ADD salary NUMBER(10,2);
```

b) Copy employee as Emp:

```
CREATE TABLE Emp AS SELECT * FROM employee;
```

c) Delete 2nd row:

```
DELETE FROM employee WHERE SNo = 2;
```

d) Drop the table:

```
DROP TABLE Emp;
```

e) Trigger for automatic updation (audit a salary change):

```
CREATE TABLE emp_audit (  
    SNo INT, old_sal NUMBER, new_sal NUMBER, changed_on DATE);  
  
CREATE OR REPLACE TRIGGER trg_sal_update  
AFTER UPDATE OF salary ON employee  
FOR EACH ROW  
BEGIN  
    INSERT INTO emp_audit  
    VALUES (:OLD.SNo, :OLD.salary, :NEW.salary, SYSDATE);  
END;  
/
```

SAMPLE OUTPUT:

```
Table employee created.  
Table altered.          -- salary column added  
Table Emp created.  
1 row deleted.  
Table Emp dropped.  
Trigger trg_sal_update created.  
  
UPDATE employee SET salary = 45000 WHERE SNo = 1;  
SELECT * FROM emp_audit;  
SNO  OLD_SAL  NEW_SAL  CHANGED_ON  
---  -  
1    40000   45000   31-MAY-26
```

RESULT:

Thus all DDL operations and the automatic-updation trigger were executed successfully.

3. EMPLOYEE table – aggregate, ordering and trigger

AIM:

To create an EMPLOYEE table and demonstrate GROUP BY, aggregate functions, ORDER BY, column RENAME and an insert trigger.

TABLE STRUCTURE:

EMPLOYEE
<u>Emp_No</u>
Emp_Name
Emp_dept
Job
Mgr
Sal

SQL QUERIES:

```
CREATE TABLE Employee (  
    Emp_No    INT PRIMARY KEY,  
    Emp_Name  VARCHAR(30),  
    Emp_dept  VARCHAR(20),  
    Job       VARCHAR(20),  
    Mgr       INT,  
    Sal       NUMBER(10,2));
```

a) Employees of a department with salary (GROUP BY):

```
SELECT Emp_dept, Emp_Name, SUM(Sal) AS Salary  
FROM   Employee  
WHERE  Emp_dept = 'CSE'  
GROUP BY Emp_dept, Emp_Name;
```

b) Lowest paid employee under each department:

```
SELECT Emp_dept, Emp_Name, Sal  
FROM   Employee  
WHERE  (Emp_dept, Sal) IN (  
        SELECT Emp_dept, MIN(Sal)  
        FROM Employee GROUP BY Emp_dept);
```

c) Employee names in descending order:

```
SELECT Emp_Name FROM Employee ORDER BY Emp_Name DESC;
```

d) Rename a column using ALTER:

```
ALTER TABLE Employee RENAME COLUMN Job TO Designation;
```

e) Insert row using a trigger (mirror inserts into a log):

```
CREATE TABLE emp_log (Emp_No INT, Emp_Name VARCHAR(30), ins_on DATE);  
  
CREATE OR REPLACE TRIGGER trg_emp_insert  
AFTER INSERT ON Employee  
FOR EACH ROW  
BEGIN  
    INSERT INTO emp_log VALUES (:NEW.Emp_No, :NEW.Emp_Name, SYSDATE);  
END;  
/
```

SAMPLE OUTPUT:

```
b)  
EMP_DEPT  EMP_NAME  SAL  
-----  -  
-----  -
```

CSE	Ravi	32000
ECE	Meena	30000

c)

EMP_NAME

Ravi

Meena

Arun

RESULT:

Thus aggregate, ordering, rename and trigger operations were executed successfully.

4. DEPARTMENTS & EMPLOYEES – DCL, TCL and procedure

AIM:

To demonstrate GRANT, REVOKE (DCL), SAVEPOINT (TCL) and a user-defined procedure on the DEPARTMENTS and EMPLOYEES tables.

E-R DIAGRAM:

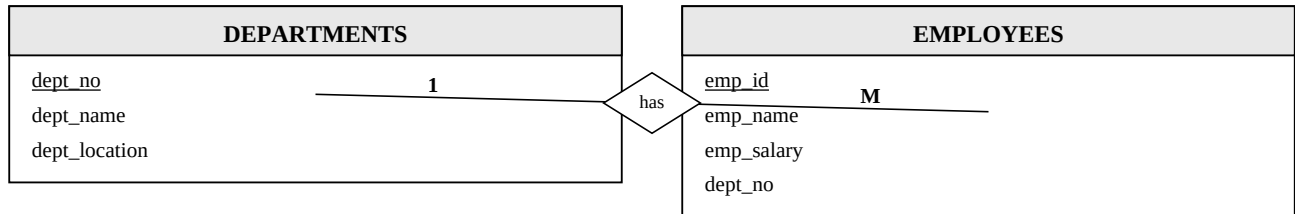


TABLE CREATION:

```
CREATE TABLE Departments (  
  dept_no      INT PRIMARY KEY,  
  dept_name    VARCHAR(30),  
  dept_location VARCHAR(30));  
  
CREATE TABLE Employees (  
  emp_id      INT PRIMARY KEY,  
  emp_name    VARCHAR(30),  
  emp_salary  NUMBER(10,2),  
  dept_no     INT REFERENCES Departments(dept_no));
```

SQL QUERIES:

a) Grant some privileges:

```
GRANT SELECT, INSERT ON Employees TO dept_user;
```

b) Revoke all privileges:

```
REVOKE ALL ON Employees FROM dept_user;
```

c) Revoke some privileges:

```
REVOKE INSERT ON Employees FROM dept_user;
```

d) Savepoint:

```
INSERT INTO Departments VALUES (10, 'CSE', 'Chennai');  
SAVEPOINT sp1;  
INSERT INTO Departments VALUES (20, 'ECE', 'Madurai');  
ROLLBACK TO sp1;      -- undoes only the ECE insert  
COMMIT;
```

e) User-defined procedure (raise salary of a department):

```
CREATE OR REPLACE PROCEDURE raise_salary(  
  p_dept IN INT, p_pct IN NUMBER) IS  
BEGIN  
  UPDATE Employees  
  SET   emp_salary = emp_salary + emp_salary * p_pct/100  
  WHERE dept_no = p_dept;  
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');  
END;  
/  
  
EXEC raise_salary(10, 10);
```

SAMPLE OUTPUT:

```
Grant succeeded.  
Revoke succeeded.
```



```
Rollback complete.  
Procedure raise_salary created.  
2 rows updated.  
PL/SQL procedure successfully completed.
```

RESULT:

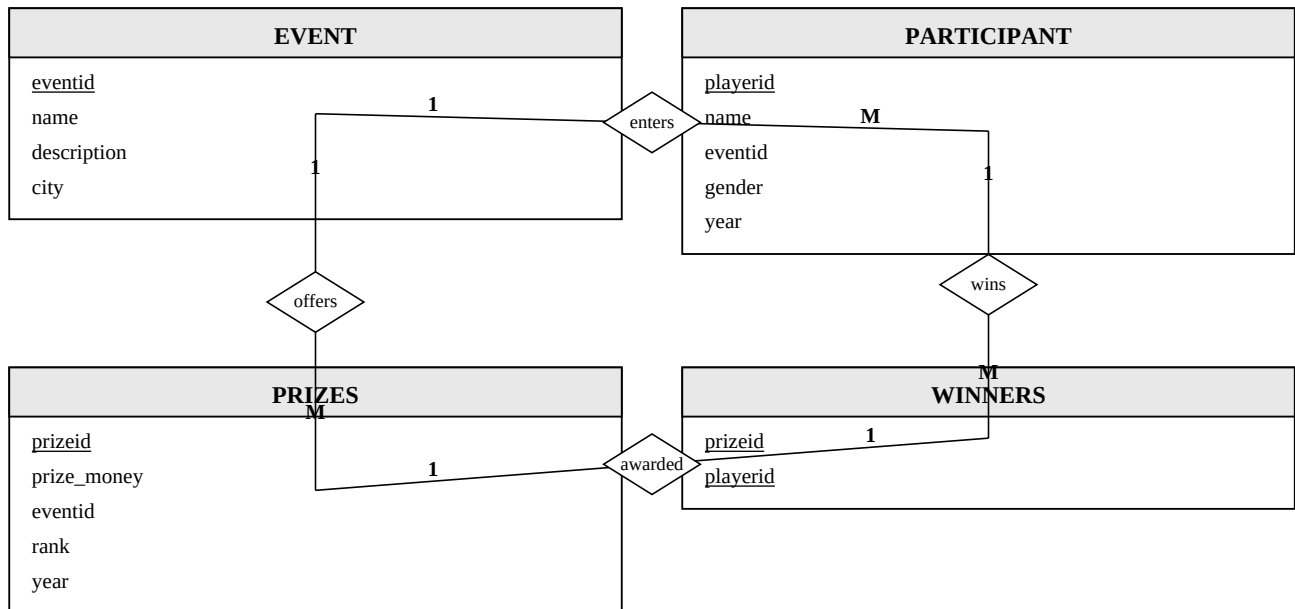
Thus DCL, TCL and the user-defined procedure were executed successfully.

5. EVENT / PARTICIPANT / PRIZES / WINNERS database

AIM:

To design the EVENT, PARTICIPANT, PRIZES and WINNERS tables with appropriate keys and constraints and to demonstrate a check constraint, complex queries, a non-updatable view and an auto-prize trigger.

E-R DIAGRAM:



a) TABLE CREATION WITH KEYS AND CHECK:

```

CREATE TABLE Event (
    eventid INT PRIMARY KEY,
    name VARCHAR(40),
    description VARCHAR(80),
    city VARCHAR(30));

CREATE TABLE Participant (
    playerid VARCHAR(10) PRIMARY KEY
    CHECK (REGEXP_LIKE(playerid, '[0-9]')), -- (b) >=1 digit
    name VARCHAR(40),
    eventid INT REFERENCES Event(eventid),
    gender CHAR(1) CHECK (gender IN ('M', 'F')),
    year INT);

CREATE TABLE Prizes (
    prizeid INT PRIMARY KEY,
    prize_money NUMBER(10,2),
    eventid INT REFERENCES Event(eventid),
    rank INT,
    year INT);

CREATE TABLE Winners (
    prizeid INT REFERENCES Prizes(prizeid),
    playerid VARCHAR(10) REFERENCES Participant(playerid),
    PRIMARY KEY (prizeid, playerid));
  
```

SQL QUERIES:

c) Events where ALL prize winners are female:

```

SELECT DISTINCT e.name
FROM Event e JOIN Prizes pz ON e.eventid = pz.eventid
  
```

```

        JOIN Winners w ON pz.prizeid = w.prizeid
        JOIN Participant p ON w.playerid = p.playerid
WHERE e.eventid NOT IN (
    SELECT pz2.eventid
    FROM Prizes pz2 JOIN Winners w2 ON pz2.prizeid = w2.prizeid
        JOIN Participant p2 ON w2.playerid = p2.playerid
    WHERE p2.gender <> 'F');

```

d) Non-updatable view of 1st-prize winners with event names:

```

CREATE VIEW first_prize_winners AS
SELECT p.name AS player_name, e.name AS event_name
FROM Prizes pz JOIN Winners w ON pz.prizeid = w.prizeid
        JOIN Participant p ON w.playerid = p.playerid
        JOIN Event e ON pz.eventid = e.eventid
WHERE pz.rank = 1
WITH READ ONLY;      -- makes the view non-updatable

```

e) Trigger: auto-create 3 prizes for every new event:

```

CREATE OR REPLACE TRIGGER trg_auto_prizes
AFTER INSERT ON Event
FOR EACH ROW
BEGIN
    INSERT INTO Prizes VALUES (seq_prize.NEXTVAL,1500,:NEW.eventid,1,NULL);
    INSERT INTO Prizes VALUES (seq_prize.NEXTVAL,1000,:NEW.eventid,2,NULL);
    INSERT INTO Prizes VALUES (seq_prize.NEXTVAL, 500,:NEW.eventid,3,NULL);
END;
/

```

SAMPLE OUTPUT:

```

c)
NAME
-----
Carrom

d)
PLAYER_NAME  EVENT_NAME
-----
Divya        Chess
Karan        Carrom

e) After  INSERT INTO Event VALUES (5,'Quiz','GK quiz','Chennai');
SELECT prizeid, prize_money, rank FROM Prizes WHERE eventid = 5;
PRIZEID  PRIZE_MONEY  RANK
-----
11        1500        1
12        1000        2
13         500        3

```

RESULT:

Thus the events database with constraints, view and trigger was implemented successfully.

6. MOVIE database

AIM:

To create the ACTOR, DIRECTOR, MOVIES and MOVIE_CAST tables and execute join queries, a view and a user-defined function.

E-R DIAGRAM:

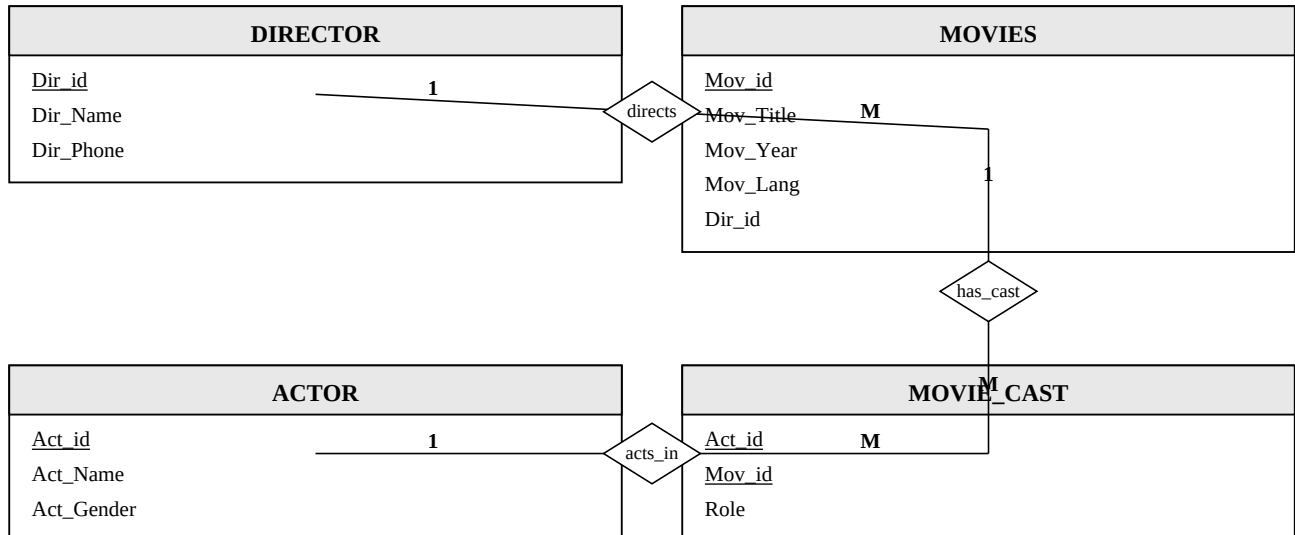


TABLE CREATION:

```
CREATE TABLE Actor (
  Act_id INT PRIMARY KEY,
  Act_Name VARCHAR(40),
  Act_Gender CHAR(1));

CREATE TABLE Director (
  Dir_id INT PRIMARY KEY,
  Dir_Name VARCHAR(40),
  Dir_Phone VARCHAR(15));

CREATE TABLE Movies (
  Mov_id INT PRIMARY KEY,
  Mov_Title VARCHAR(60),
  Mov_Year INT,
  Mov_Lang VARCHAR(20),
  Dir_id INT REFERENCES Director(Dir_id));

CREATE TABLE Movie_Cast (
  Act_id INT REFERENCES Actor(Act_id),
  Mov_id INT REFERENCES Movies(Mov_id),
  Role VARCHAR(30),
  PRIMARY KEY (Act_id, Mov_id));
```

SQL QUERIES:

a) Titles of all movies directed by 'Hitchcock':

```
SELECT m.Mov_Title
FROM Movies m JOIN Director d ON m.Dir_id = d.Dir_id
WHERE d.Dir_Name = 'Hitchcock';
```

b) Movies where one or more actors acted in 2+ movies:

```
SELECT DISTINCT m.Mov_Title
FROM Movies m JOIN Movie_Cast mc ON m.Mov_id = mc.Mov_id
WHERE mc.Act_id IN (
  SELECT Act_id FROM Movie_Cast
```

```
GROUP BY Act_id HAVING COUNT(Mov_id) >= 2);
```

c) Actors in a movie before 2010 and after 2015 (JOIN):

```
SELECT DISTINCT a.Act_Name
FROM   Actor a JOIN Movie_Cast c1 ON a.Act_id = c1.Act_id
        JOIN Movies m1 ON c1.Mov_id = m1.Mov_id
        JOIN Movie_Cast c2 ON a.Act_id = c2.Act_id
        JOIN Movies m2 ON c2.Mov_id = m2.Mov_id
WHERE  m1.Mov_Year < 2010 AND m2.Mov_Year > 2015;
```

d) View of movies with a particular actor and director:

```
CREATE VIEW actor_movies AS
SELECT a.Act_Name, m.Mov_Title, d.Dir_Name
FROM   Actor a JOIN Movie_Cast mc ON a.Act_id = mc.Act_id
        JOIN Movies m ON mc.Mov_id = m.Mov_id
        JOIN Director d ON m.Dir_id = d.Dir_id
WHERE  a.Act_Name = 'Kamal';
```

e) User-defined function (count movies of an actor):

```
CREATE OR REPLACE FUNCTION movie_count(p_act IN INT)
RETURN NUMBER IS v_cnt NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_cnt FROM Movie_Cast WHERE Act_id = p_act;
    RETURN v_cnt;
END;
/

SELECT movie_count(1) AS total_movies FROM dual;
```

SAMPLE OUTPUT:

```
a)
MOV_TITLE
-----
Psycho
The Birds

e)
TOTAL_MOVIES
-----
3
```

RESULT:

Thus the Movie database with joins, view and function was implemented successfully.

7. COLLEGE database

AIM:

To create the STUDENT, SUBJECT and MARKS tables and to compute counts, the final mark (average of best two tests), categorize students, build a view and write a procedure.

E-R DIAGRAM:

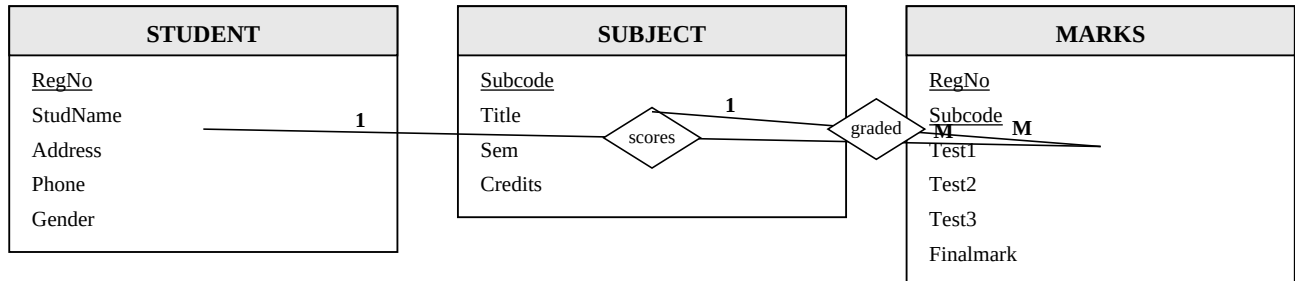


TABLE CREATION:

```
CREATE TABLE Student (
    RegNo    VARCHAR(12) PRIMARY KEY,
    StudName VARCHAR(40), Address VARCHAR(60),
    Phone    VARCHAR(15), Gender CHAR(1),
    Section  CHAR(1), Sem INT);

CREATE TABLE Subject (
    Subcode VARCHAR(10) PRIMARY KEY,
    Title   VARCHAR(40), Sem INT, Credits INT);

CREATE TABLE Marks (
    RegNo    VARCHAR(12) REFERENCES Student(RegNo),
    Subcode  VARCHAR(10) REFERENCES Subject(Subcode),
    Test1 INT, Test2 INT, Test3 INT, Finalmark INT,
    PRIMARY KEY (RegNo, Subcode));
```

SQL QUERIES:

a) Count of male and female students per semester and section:

```
SELECT Sem, Section, Gender, COUNT(*) AS Total
FROM   Student
GROUP BY Sem, Section, Gender
ORDER BY Sem, Section;
```

b) Finalmark = average of best two test marks:

```
UPDATE Marks
SET Finalmark = (Test1 + Test2 + Test3
                - LEAST(Test1,Test2,Test3)) / 2;
```

c) Categorize students:

```
SELECT RegNo, Subcode, Finalmark,
CASE WHEN Finalmark BETWEEN 81 AND 100 THEN 'Outstanding'
     WHEN Finalmark BETWEEN 51 AND 80  THEN 'Average'
     ELSE 'Weak'
END AS CAT
FROM Marks;
```

d) View of Test3 marks of a particular student:

```
CREATE VIEW test3_view AS
SELECT m.RegNo, m.Subcode, m.Test3
FROM   Marks m
WHERE  m.RegNo = '950021CS01';
```

e) Procedure (display result of a student):

```
CREATE OR REPLACE PROCEDURE show_result(p_reg IN VARCHAR2) IS
BEGIN
  FOR r IN (SELECT Subcode, Finalmark FROM Marks
            WHERE RegNo = p_reg) LOOP
    DBMS_OUTPUT.PUT_LINE(r.Subcode||' : '||r.Finalmark);
  END LOOP;
END;
/

EXEC show_result('950021CS01');
```

SAMPLE OUTPUT:

a)

SEM	SECTION	GENDER	TOTAL
4	A	M	28
4	A	F	22

c)

REGNO	SUBCODE	FINALMARK	CAT
950021CS01	CS1256	88	Outstanding
950021CS02	CS1256	64	Average
950021CS03	CS1256	42	Weak

RESULT:

Thus the College database operations were executed successfully.

8. BANK table – SELECT variants and trigger

AIM:

To create a BANK table and demonstrate SELECT with WHERE, comparison operators, BETWEEN, an UPDATE and a low-balance trigger.

TABLE STRUCTURE:

BANK
SNo
Cust_Name
<u>Acc_No</u>
Balance
Branch

SQL QUERIES:

```
CREATE TABLE Bank (  
  SNo      INT,  
  Cust_Name VARCHAR(30),  
  Acc_No   INT PRIMARY KEY,  
  Balance  NUMBER(12,2),  
  Branch   VARCHAR(30));
```

a) Select with WHERE clause:

```
SELECT * FROM Bank WHERE Branch = 'Anna Nagar';
```

b) Select with comparison operator:

```
SELECT Cust_Name, Balance FROM Bank WHERE Balance > 5000;
```

c) Update balance in the second row:

```
UPDATE Bank SET Balance = Balance + 2000 WHERE SNo = 2;
```

d) Select with BETWEEN on balance:

```
SELECT * FROM Bank WHERE Balance BETWEEN 1000 AND 10000;
```

e) Trigger when balance is below 1000:

```
CREATE OR REPLACE TRIGGER trg_low_balance  
BEFORE INSERT OR UPDATE ON Bank  
FOR EACH ROW  
BEGIN  
  IF :NEW.Balance < 1000 THEN  
    RAISE_APPLICATION_ERROR(-20001,  
      'Balance cannot be below Rs.1000');  
  END IF;  
END;  
/
```

SAMPLE OUTPUT:

```
b)  
CUST_NAME  BALANCE  
-----  
Anand      8000  
Banu       12000  
  
e)  
UPDATE Bank SET Balance = 500 WHERE SNo = 1;  
ERROR at line 1:  
ORA-20001: Balance cannot be below Rs.1000
```

RESULT:

Thus the Bank table operations and the low-balance trigger were executed successfully.

9. ACCOUNT table – queries, ALTER and procedure

AIM:

To create an ACCOUNT table with a suitable primary key and execute conditional queries, an ALTER and a procedure.

TABLE STRUCTURE:

ACCOUNT
<u>Account_No</u>
Cust_Name
Branch_Name
Account_Balance
Account_Type

SQL QUERIES:

```
CREATE TABLE Account (  
    Account_No      INT PRIMARY KEY,  
    Cust_Name       VARCHAR(30),  
    Branch_Name     VARCHAR(30),  
    Account_Balance NUMBER(12,2),  
    Account_Type    VARCHAR(15));
```

a) Cust Name and Account No for a branch:

```
SELECT Cust_Name, Account_No  
FROM   Account WHERE Branch_Name = 'T Nagar';
```

b) Names and account types where balance > 10,000:

```
SELECT Cust_Name, Account_Type  
FROM   Account WHERE Account_Balance > 10000;
```

c) Add column Cust Date of Birth:

```
ALTER TABLE Account ADD Cust_Date_of_Birth DATE;
```

d) Account No, Cust Name, Branch where balance < 1,000:

```
SELECT Account_No, Cust_Name, Branch_Name  
FROM   Account WHERE Account_Balance < 1000;
```

e) Procedure (total balance of a branch):

```
CREATE OR REPLACE PROCEDURE branch_total(p_br IN VARCHAR2) IS  
    v_tot NUMBER;  
BEGIN  
    SELECT SUM(Account_Balance) INTO v_tot  
    FROM Account WHERE Branch_Name = p_br;  
    DBMS_OUTPUT.PUT_LINE('Total balance = '||v_tot);  
END;  
/  
  
EXEC branch_total('T Nagar');
```

SAMPLE OUTPUT:

```
b)  
CUST_NAME  ACCOUNT_TYPE  
-----  
Kumar      Savings  
Latha      Current
```

```
e)  
Total balance = 47000  
PL/SQL procedure successfully completed.
```

RESULT:

Thus the Account table operations were executed successfully.

10. CUSTOMER & ORDERS – queries and function

AIM:

To create CUSTOMER and ORDERS tables and execute join queries, an ALTER and a user-defined function.

E-R DIAGRAM:

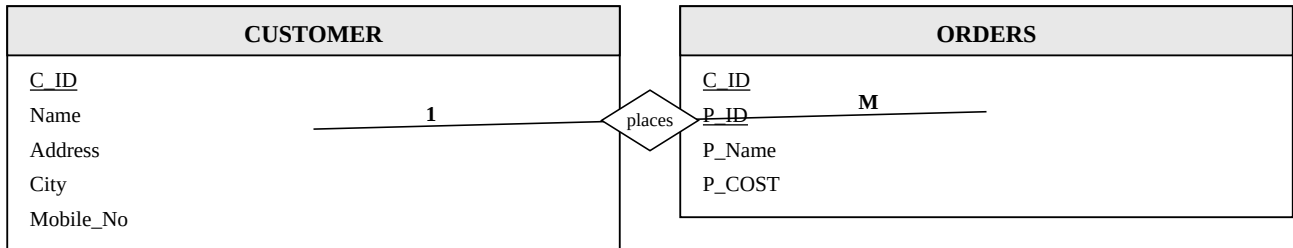


TABLE CREATION:

```
CREATE TABLE Customer (
  C_ID      INT PRIMARY KEY,
  Name      VARCHAR(30), Address VARCHAR(60),
  City      VARCHAR(30), Mobile_No VARCHAR(15));

CREATE TABLE Orders (
  C_ID      INT REFERENCES Customer(C_ID),
  P_ID      INT, P_Name VARCHAR(30), P_COST NUMBER(10,2),
  PRIMARY KEY (C_ID, P_ID));
```

SQL QUERIES:

a) Names and addresses of customers who ordered products costing more than 500:

```
SELECT DISTINCT c.Name, c.Address
FROM   Customer c JOIN Orders o ON c.C_ID = o.C_ID
WHERE  o.P_COST > 500;
```

b) Products ordered whose cost is 1,000 or more:

```
SELECT DISTINCT P_Name FROM Orders WHERE P_COST >= 1000;
```

c) Product names ordered by customers of City = Delhi:

```
SELECT DISTINCT o.P_Name
FROM   Customer c JOIN Orders o ON c.C_ID = o.C_ID
WHERE  c.City = 'Delhi';
```

d) Add column Email id:

```
ALTER TABLE Customer ADD Email_id VARCHAR(40);
```

e) User-defined function (total spend of a customer):

```
CREATE OR REPLACE FUNCTION total_spend(p_cid IN INT)
RETURN NUMBER IS v NUMBER;
BEGIN
  SELECT NVL(SUM(P_COST),0) INTO v FROM Orders WHERE C_ID = p_cid;
  RETURN v;
END;
/

SELECT total_spend(1) AS spend FROM dual;
```

SAMPLE OUTPUT:

```
a)
NAME      ADDRESS
-----
Ramesh    12 MG Road
```

```
e)
SPEND
-----
3200
```

RESULT:

Thus the Customer and Orders operations were executed successfully.

11. SALESMAN / CUSTOMER / ORDERS database

AIM:

To create the SALESMAN, CUSTOMER and ORDERS tables and execute grouping, UNION, a view, a cascading delete and a trigger.

E-R DIAGRAM:

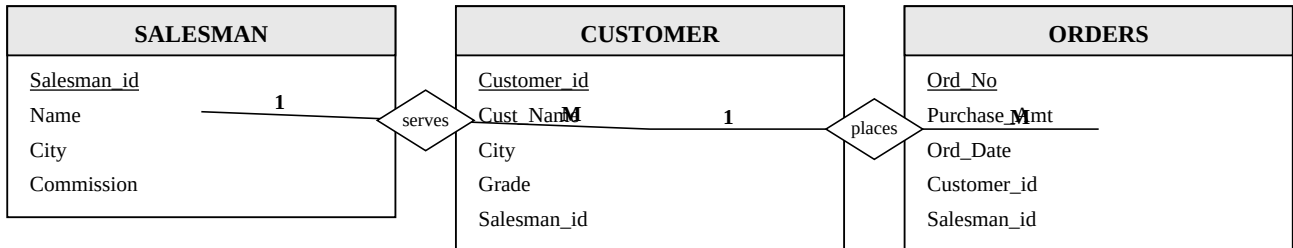


TABLE CREATION:

```
CREATE TABLE Salesman (
    Salesman_id INT PRIMARY KEY,
    Name VARCHAR(30), City VARCHAR(30), Commission NUMBER(5,2));

CREATE TABLE Customer (
    Customer_id INT PRIMARY KEY,
    Cust_Name VARCHAR(30), City VARCHAR(30), Grade INT,
    Salesman_id INT REFERENCES Salesman(Salesman_id)
    ON DELETE CASCADE);

CREATE TABLE Orders (
    Ord_No INT PRIMARY KEY, Purchase_Amt NUMBER(12,2),
    Ord_Date DATE,
    Customer_id INT REFERENCES Customer(Customer_id),
    Salesman_id INT REFERENCES Salesman(Salesman_id)
    ON DELETE CASCADE);
```

SQL QUERIES:

a) Salesmen with more than one customer:

```
SELECT s.Salesman_id, s.Name
FROM Salesman s JOIN Customer c ON s.Salesman_id = c.Salesman_id
GROUP BY s.Salesman_id, s.Name
HAVING COUNT(c.Customer_id) > 1;
```

b) All salesmen, indicating who have customers in their cities (UNION):

```
SELECT s.Name, 'Has customer in city' AS Status
FROM Salesman s
WHERE EXISTS (SELECT 1 FROM Customer c
              WHERE c.Salesman_id = s.Salesman_id
              AND c.City = s.City)

UNION

SELECT s.Name, 'No customer in city'
FROM Salesman s
WHERE NOT EXISTS (SELECT 1 FROM Customer c
                  WHERE c.Salesman_id = s.Salesman_id
                  AND c.City = s.City);
```

c) View: salesman with the highest order of a day:

```
CREATE VIEW top_salesman_day AS
SELECT o.Ord_Date, o.Salesman_id, o.Purchase_Amt
FROM Orders o
WHERE o.Purchase_Amt = (
    SELECT MAX(o2.Purchase_Amt) FROM Orders o2
    WHERE o2.Ord_Date = o.Ord_Date);
```

```
WHERE o2.Ord_Date = o.Ord_Date);
```

d) Delete salesman 1000 (orders also deleted):

```
DELETE FROM Salesman WHERE Salesman_id = 1000;  
-- ON DELETE CASCADE removes that salesman's customers and orders.
```

e) Trigger (log every new order):

```
CREATE TABLE order_log (Ord_No INT, logged_on DATE);  
  
CREATE OR REPLACE TRIGGER trg_order_ins  
AFTER INSERT ON Orders  
FOR EACH ROW  
BEGIN  
    INSERT INTO order_log VALUES (:NEW.Ord_No, SYSDATE);  
END;  
/
```

SAMPLE OUTPUT:

```
a)  
SALESMAN_ID  NAME  
-----  
5001         James  
  
b)  
NAME        STATUS  
-----  
James      Has customer in city  
Nair       No customer in city
```

RESULT:

Thus the Salesman/Customer/Orders operations were executed successfully.

12. GUI-based Inventory Management – EMart Grocery Shop

AIM:

To develop a GUI-based Inventory Management application for an EMart Grocery Shop using Python (Tkinter) with a SQLite backend, incorporating insert, view, update, delete and search database features.

DATABASE DESIGN:

```
CREATE TABLE inventory (  
    item_id    INTEGER PRIMARY KEY AUTOINCREMENT,  
    item_name  TEXT NOT NULL,  
    category   TEXT,  
    quantity  INTEGER,  
    price     REAL);
```

SOURCE CODE (emart_inventory.py):

```
import tkinter as tk  
from tkinter import ttk, messagebox  
import sqlite3  
  
def db():  
    c = sqlite3.connect('emart.db')  
    c.execute('''CREATE TABLE IF NOT EXISTS inventory(  
        item_id INTEGER PRIMARY KEY AUTOINCREMENT,  
        item_name TEXT NOT NULL, category TEXT,  
        quantity INTEGER, price REAL)''')  
    return c  
  
class EMart:  
    def __init__(self, root):  
        root.title('EMart Grocery - Inventory'); root.geometry('640x460')  
        self.name = tk.StringVar(); self.cat = tk.StringVar()  
        self.qty = tk.StringVar(); self.price = tk.StringVar()  
        self.sel = None  
        frm = tk.LabelFrame(root, text='Item Details'); frm.pack(fill='x', padx=8, pady=6)  
        tk.Label(frm, text='Name').grid(row=0, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.name).grid(row=0, column=1)  
        tk.Label(frm, text='Category').grid(row=0, column=2)  
        tk.Entry(frm, textvariable=self.cat).grid(row=0, column=3)  
        tk.Label(frm, text='Quantity').grid(row=1, column=0)  
        tk.Entry(frm, textvariable=self.qty).grid(row=1, column=1)  
        tk.Label(frm, text='Price').grid(row=1, column=2)  
        tk.Entry(frm, textvariable=self.price).grid(row=1, column=3)  
        bf = tk.Frame(root); bf.pack(pady=4)  
        tk.Button(bf, text='Add', width=10, command=self.add).pack(side='left', padx=3)  
        tk.Button(bf, text='Update', width=10, command=self.update).pack(side='left', padx=3)  
        tk.Button(bf, text='Delete', width=10, command=self.delete).pack(side='left', padx=3)  
        tk.Button(bf, text='Clear', width=10, command=self.clear).pack(side='left', padx=3)  
        cols = ('id', 'name', 'cat', 'qty', 'price')  
        self.tree = ttk.Treeview(root, columns=cols, show='headings', height=12)  
        for c in cols:  
            self.tree.heading(c, text=c.title()); self.tree.column(c, width=110)  
        self.tree.pack(fill='both', expand=True, padx=8, pady=6)  
        self.tree.bind('<<TreeviewSelect>>', self.pick)  
        self.show()  
  
    def add(self):  
        if not self.name.get():  
            messagebox.showwarning('!', 'Name required'); return  
        c = db(); c.execute('INSERT INTO inventory(item_name,category,quantity,price)'  
            ' VALUES(?,?,?,?)',
```



```

        (self.name.get(), self.cat.get(),
         self.qty.get(), self.price.get()))
    c.commit(); c.close(); self.clear(); self.show()

def update(self):
    if not self.sel: return
    c = db(); c.execute('UPDATE inventory SET item_name=?,category=?,'
                        'quantity=?,price=? WHERE item_id=?',
                        (self.name.get(), self.cat.get(), self.qty.get(),
                         self.price.get(), self.sel))
    c.commit(); c.close(); self.clear(); self.show()

def delete(self):
    if not self.sel: return
    c = db(); c.execute('DELETE FROM inventory WHERE item_id=?', (self.sel,))
    c.commit(); c.close(); self.clear(); self.show()

def pick(self, e):
    r = self.tree.focus()
    if not r: return
    v = self.tree.item(r)['values']
    self.sel = v[0]; self.name.set(v[1]); self.cat.set(v[2])
    self.qty.set(v[3]); self.price.set(v[4])

def clear(self):
    self.sel = None
    for x in (self.name, self.cat, self.qty, self.price): x.set('')

def show(self):
    self.tree.delete(*self.tree.get_children())
    c = db()
    for row in c.execute('SELECT * FROM inventory'):
        self.tree.insert('', 'end', values=row)
    c.close()

if __name__ == '__main__':
    root = tk.Tk(); EMart(root); root.mainloop()

```

SAMPLE OUTPUT:

Window 'EMart Grocery - Inventory' opens with entry fields and a grid.

ID	NAME	CAT	QTY	PRICE
1	Rice	Grocery	50	62.0
2	Sugar	Grocery	30	42.5
3	Oil	Grocery	25	135.0

Add / Update / Delete buttons modify the inventory table live.

RESULT:

Thus the GUI-based EMart inventory application with full database features was developed successfully.

13. XML database for student profile with XML Schema validation

AIM:

To create an XML database for a student profile and validate it against an XML Schema Definition (XSD).

XML DOCUMENT (student.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student regno="950021CS01">
    <name>Arun Kumar</name>
    <dept>CSE</dept>
    <semester>4</semester>
    <cgpa>8.5</cgpa>
    <email>arun@dmice.ac.in</email>
  </student>
  <student regno="950021CS02">
    <name>Divya R</name>
    <dept>CSE</dept>
    <semester>4</semester>
    <cgpa>9.1</cgpa>
    <email>divya@dmice.ac.in</email>
  </student>
</students>
```

XML SCHEMA (student.xsd):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="students">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="student" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="dept" type="xs:string"/>
              <xs:element name="semester" type="xs:integer"/>
              <xs:element name="cgpa" type="xs:decimal"/>
              <xs:element name="email" type="xs:string"/>
            </xs:sequence>
            <xs:attribute name="regno" type="xs:string"
              use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

VALIDATION (Python lxml):

```
from lxml import etree
schema = etree.XMLSchema(etree.parse('student.xsd'))
doc = etree.parse('student.xml')
print('Valid' if schema.validate(doc) else schema.error_log)
```

SAMPLE OUTPUT:

```
Valid
(The student.xml document conforms to student.xsd.)
```

RESULT:

Thus the XML student-profile database was created and validated against the XML schema successfully.

14. GUI-based Cop Friendly App – Eseva

AIM:

To develop a GUI-based Cop Friendly 'Eseva' application using Python (Tkinter) with a SQLite backend for registering and managing public complaints (FIR / grievances), incorporating insert, view, update and search features.

DATABASE DESIGN:

```
CREATE TABLE complaints (  
    comp_id    INTEGER PRIMARY KEY AUTOINCREMENT,  
    citizen    TEXT NOT NULL,  
    phone      TEXT,  
    category   TEXT,  
    details    TEXT,  
    status     TEXT DEFAULT 'Open');
```

SOURCE CODE (eseva.py):

```
import tkinter as tk  
from tkinter import ttk, messagebox  
import sqlite3  
  
def db():  
    c = sqlite3.connect('eseva.db')  
    c.execute('''CREATE TABLE IF NOT EXISTS complaints(  
        comp_id INTEGER PRIMARY KEY AUTOINCREMENT,  
        citizen TEXT NOT NULL, phone TEXT, category TEXT,  
        details TEXT, status TEXT DEFAULT 'Open')''')  
    return c  
  
class Eseva:  
    def __init__(self, root):  
        root.title('Eseva - Cop Friendly App'); root.geometry('680x500')  
        self.cz = tk.StringVar(); self.ph = tk.StringVar()  
        self.cat = tk.StringVar(value='Theft'); self.st = tk.StringVar(value='Open')  
        self.sel = None  
        frm = tk.LabelFrame(root, text='Register Complaint'); frm.pack(fill='x', padx=8, pady=4)  
        tk.Label(frm, text='Citizen').grid(row=0, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.cz).grid(row=0, column=1)  
        tk.Label(frm, text='Phone').grid(row=0, column=2)  
        tk.Entry(frm, textvariable=self.ph).grid(row=0, column=3)  
        tk.Label(frm, text='Category').grid(row=1, column=0)  
        ttk.Combobox(frm, textvariable=self.cat,  
            values=['Theft','Traffic','Missing','Cyber','Other']).grid(row=1, column=1)  
        tk.Label(frm, text='Status').grid(row=1, column=2)  
        ttk.Combobox(frm, textvariable=self.st,  
            values=['Open','In-Progress','Closed']).grid(row=1, column=3)  
        tk.Label(frm, text='Details').grid(row=2, column=0)  
        self.det = tk.Text(frm, width=58, height=3)  
        self.det.grid(row=2, column=1, columnspan=3, pady=4)  
        bf = tk.Frame(root); bf.pack(pady=4)  
        tk.Button(bf, text='File', width=10, command=self.add).pack(side='left', padx=3)  
        tk.Button(bf, text='Update', width=10, command=self.update).pack(side='left', padx=3)  
        tk.Button(bf, text='Delete', width=10, command=self.delete).pack(side='left', padx=3)  
        tk.Button(bf, text='Clear', width=10, command=self.clear).pack(side='left', padx=3)  
        cols = ('id', 'citizen', 'phone', 'category', 'status')  
        self.tree = ttk.Treeview(root, columns=cols, show='headings', height=11)  
        for c in cols:  
            self.tree.heading(c, text=c.title()); self.tree.column(c, width=120)  
        self.tree.pack(fill='both', expand=True, padx=8, pady=6)  
        self.tree.bind('<<TreeviewSelect>>', self.pick)  
        self.show()
```

```

def add(self):
    if not self.cz.get():
        messagebox.showwarning('!', 'Citizen name required'); return
    c = db(); c.execute('INSERT INTO complaints'
        '(citizen,phone,category,details,status) VALUES(?,?,?,?,?)',
        (self.cz.get(), self.ph.get(), self.cat.get(),
        self.det.get('1.0','end').strip(), self.st.get()))
    c.commit(); c.close(); self.clear(); self.show()

def update(self):
    if not self.sel: return
    c = db(); c.execute('UPDATE complaints SET citizen=?,phone=?, '
        'category=?,details=?,status=? WHERE comp_id=?',
        (self.cz.get(), self.ph.get(), self.cat.get(),
        self.det.get('1.0','end').strip(), self.st.get(), self.sel))
    c.commit(); c.close(); self.clear(); self.show()

def delete(self):
    if not self.sel: return
    c = db(); c.execute('DELETE FROM complaints WHERE comp_id=?', (self.sel,))
    c.commit(); c.close(); self.clear(); self.show()

def pick(self, e):
    r = self.tree.focus()
    if not r: return
    v = self.tree.item(r)['values']
    self.sel = v[0]; self.cz.set(v[1]); self.ph.set(v[2])
    self.cat.set(v[3]); self.st.set(v[4])

def clear(self):
    self.sel = None
    self.cz.set(''); self.ph.set(''); self.det.delete('1.0','end')

def show(self):
    self.tree.delete(*self.tree.get_children())
    c = db()
    for row in c.execute('SELECT comp_id,citizen,phone,category,status'
        ' FROM complaints'):
        self.tree.insert('', 'end', values=row)
    c.close()

if __name__ == '__main__':
    root = tk.Tk(); Eseva(root); root.mainloop()

```

SAMPLE OUTPUT:

ID	CITIZEN	PHONE	CATEGORY	STATUS
1	Suresh	9840012345	Theft	Open
2	Meena	9952233445	Traffic	In-Progress
3	Karthik	9001122334	Cyber	Closed

RESULT:

Thus the GUI-based Eseva cop-friendly application with full database features was developed successfully.

15. EMPLOYEE database – triggers for INSERT, DELETE, UPDATE

AIM:

To create an Employee database and write SQL triggers that audit insert, delete and update operations on the table.

TABLE STRUCTURE:

EMPLOYEE	EMP_AUDIT
<u>Emp_id</u> Emp_name Dept Salary	Emp_id Operation op_time

SOURCE CODE:

```
CREATE TABLE Employee (  
  Emp_id INT PRIMARY KEY,  
  Emp_name VARCHAR(30), Dept VARCHAR(20), Salary NUMBER(10,2));  
  
CREATE TABLE Emp_Audit (  
  Emp_id INT, Operation VARCHAR(10), op_time DATE);
```

Insert trigger:

```
CREATE OR REPLACE TRIGGER trg_ins  
AFTER INSERT ON Employee FOR EACH ROW  
BEGIN  
  INSERT INTO Emp_Audit VALUES (:NEW.Emp_id, 'INSERT', SYSDATE);  
END;  
/
```

Delete trigger:

```
CREATE OR REPLACE TRIGGER trg_del  
AFTER DELETE ON Employee FOR EACH ROW  
BEGIN  
  INSERT INTO Emp_Audit VALUES (:OLD.Emp_id, 'DELETE', SYSDATE);  
END;  
/
```

Update trigger:

```
CREATE OR REPLACE TRIGGER trg_upd  
AFTER UPDATE ON Employee FOR EACH ROW  
BEGIN  
  INSERT INTO Emp_Audit VALUES (:NEW.Emp_id, 'UPDATE', SYSDATE);  
END;  
/
```

SAMPLE OUTPUT:

```
INSERT INTO Employee VALUES (1,'Ravi','CSE',40000);  
UPDATE Employee SET Salary = 45000 WHERE Emp_id = 1;  
DELETE FROM Employee WHERE Emp_id = 1;  
SELECT * FROM Emp_Audit;  
EMP_ID  OPERATION  OP_TIME  
-----  
1        INSERT    31-MAY-26  
1        UPDATE    31-MAY-26  
1        DELETE    31-MAY-26
```

RESULT:

Thus the insert, delete and update triggers on the Employee database were implemented successfully.

16. SUPPLIER table – queries, ALTER, view and procedure

AIM:

To create a SUPPLIER table and demonstrate pattern matching, ALTER, ordering, a view and a procedure.

TABLE STRUCTURE:

SUPPLIER
Sup_No
Sup_Name
Item_Supplied
Item_Price
City

SQL QUERIES:

```
CREATE TABLE Supplier (  
  Sup_No          INT PRIMARY KEY,  
  Sup_Name        VARCHAR(30),  
  Item_Supplied   VARCHAR(30),  
  Item_Price      NUMBER(10,2),  
  City            VARCHAR(30));
```

a) Suppliers whose name starts with 'S':

```
SELECT Sup_No, Sup_Name FROM Supplier WHERE Sup_Name LIKE 'S%';
```

b) Add column CONTACT_NO:

```
ALTER TABLE Supplier ADD Contact_No VARCHAR(15);
```

c) Suppliers in Chennai, ascending by item price:

```
SELECT Sup_No, Sup_Name, Item_Price  
FROM Supplier  
WHERE City = 'Chennai'  
ORDER BY Item_Price ASC;
```

d) View of supplier numbers and names:

```
CREATE VIEW supplier_view AS  
SELECT Sup_No, Sup_Name FROM Supplier;
```

e) Procedure (count suppliers in a city):

```
CREATE OR REPLACE PROCEDURE sup_count(p_city IN VARCHAR2) IS  
  v NUMBER;  
BEGIN  
  SELECT COUNT(*) INTO v FROM Supplier WHERE City = p_city;  
  DBMS_OUTPUT.PUT_LINE('Suppliers in '||p_city||' = '||v);  
END;  
/  
  
EXEC sup_count('Chennai');
```

SAMPLE OUTPUT:

```
a)  
SUP_NO  SUP_NAME  
-----  
101     Saravana  
104     Senthil  
  
c)  
SUP_NO  SUP_NAME  ITEM_PRICE  
-----  
101     Saravana  10.50  
104     Senthil   12.00
```


104	Senthil	45.0
101	Saravana	80.0

e)
Suppliers in Chennai = 2

RESULT:

Thus the Supplier table operations were executed successfully.

17. GUI-based Banking System

AIM:

To develop a GUI-based Banking System using Python (Tkinter) with a SQLite backend to open accounts and perform deposit, withdrawal and balance enquiry with full database features.

DATABASE DESIGN:

```
CREATE TABLE account (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    acc_no       TEXT,  
    name         TEXT,  
    acc_type     TEXT,  
    balance      REAL);
```

SOURCE CODE (banking.py):

```
import tkinter as tk  
from tkinter import ttk, messagebox  
import sqlite3  
  
def db():  
    c = sqlite3.connect('bank.db')  
    c.execute('''CREATE TABLE IF NOT EXISTS account(  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        acc_no TEXT,  
        name TEXT,  
        acc_type TEXT,  
        balance REAL)''')  
    return c  
  
class Banking:  
    def __init__(self, root):  
        root.title('Banking System'); root.geometry('660x480')  
        self.sel = None; self.v = {}  
        self.v['acc_no'] = tk.StringVar()  
        self.v['name'] = tk.StringVar()  
        self.v['acc_type'] = tk.StringVar()  
        self.v['balance'] = tk.StringVar()  
        frm = tk.LabelFrame(root, text='Details')  
        frm.pack(fill='x', padx=8, pady=6)  
        tk.Label(frm, text='Acc No').grid(row=0, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['acc_no']).grid(row=0, column=1)  
        tk.Label(frm, text='Name').grid(row=0, column=2, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['name']).grid(row=0, column=3)  
        tk.Label(frm, text='Type').grid(row=1, column=0, padx=4, pady=4)  
        ttk.Combobox(frm, textvariable=self.v['acc_type'], values=['Savings', 'Current', 'FD']).grid(row=1, column=1)  
        tk.Label(frm, text='Balance').grid(row=1, column=2, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['balance']).grid(row=1, column=3)  
        bf = tk.Frame(root); bf.pack(pady=4)  
        tk.Button(bf, text='Add', width=9, command=self.add).pack(side='left', padx=3)  
        tk.Button(bf, text='Update', width=9, command=self.update).pack(side='left', padx=3)  
        tk.Button(bf, text='Delete', width=9, command=self.delete).pack(side='left', padx=3)  
        tk.Button(bf, text='Clear', width=9, command=self.clear).pack(side='left', padx=3)  
        cols = ('id', 'acc_no', 'name', 'acc_type', 'balance')  
        self.tree = ttk.Treeview(root, columns=cols, show='headings', height=11)  
        for c in cols:  
            self.tree.heading(c, text=c.title()); self.tree.column(c, width=100)  
        self.tree.pack(fill='both', expand=True, padx=8, pady=6)  
        self.tree.bind('<<TreeviewSelect>>', self.pick)  
        self.show()  
  
    def _vals(self):  
        return [self.v[k].get() for k in ['acc_no', 'name', 'acc_type', 'balance']]
```

```

def add(self):
    c = db(); c.execute('INSERT INTO account(acc_no,name,acc_type,balance) VALUES(?,?,?,?)
                        self._vals())
    c.commit(); c.close(); self.clear(); self.show()

def update(self):
    if not self.sel: return
    c = db(); c.execute('UPDATE account SET acc_no=?,name=?,acc_type=?,balance=? WHERE id=
                        self._vals() + [self.sel])
    c.commit(); c.close(); self.clear(); self.show()

def delete(self):
    if not self.sel: return
    c = db(); c.execute('DELETE FROM account WHERE id=?', (self.sel,))
    c.commit(); c.close(); self.clear(); self.show()

def pick(self, e):
    r = self.tree.focus()
    if not r: return
    val = self.tree.item(r)['values']; self.sel = val[0]
    for i, k in enumerate(['acc_no', 'name', 'acc_type', 'balance']):
        self.v[k].set(val[i + 1])

def clear(self):
    self.sel = None
    for k in self.v: self.v[k].set('')

def show(self):
    self.tree.delete(*self.tree.get_children())
    c = db()
    for row in c.execute('SELECT * FROM account'):
        self.tree.insert('', 'end', values=row)
    c.close()

if __name__ == '__main__':
    root = tk.Tk(); Banking(root); root.mainloop()

```

SAMPLE OUTPUT:

ID	ACC_NO	NAME	ACC_TYPE	BALANCE
1	100201	Anand	Savings	15000.0
2	100202	Beena	Current	42000.0

Add opens an account; Update reflects deposits/withdrawals.

RESULT:

Thus the GUI-based Banking System with full database features was developed successfully.

18. GUI-based Employee Pay Roll System

AIM:

To develop a GUI-based Employee Pay Roll System using Python (Tkinter) with a SQLite backend that stores employee pay components and computes net pay, with full database features.

DATABASE DESIGN:

```
CREATE TABLE payroll (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  emp_name    TEXT,
  designation TEXT,
  basic       REAL,
  hra         REAL,
  da          REAL,
  net_pay     REAL);
```

Note: $\text{net_pay} = \text{basic} + \text{hra} + \text{da}$. The application computes it on save; the SQL form of the computation is `UPDATE payroll SET net_pay = basic + hra + da;`

SOURCE CODE (payroll.py):

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3

def db():
    c = sqlite3.connect('payroll.db')
    c.execute('''CREATE TABLE IF NOT EXISTS payroll(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        emp_name TEXT,
        designation TEXT,
        basic REAL,
        hra REAL,
        da REAL,
        net_pay REAL)''')
    return c

class Payroll:
    def __init__(self, root):
        root.title('Employee Pay Roll'); root.geometry('660x480')
        self.sel = None; self.v = {}
        self.v['emp_name'] = tk.StringVar()
        self.v['designation'] = tk.StringVar()
        self.v['basic'] = tk.StringVar()
        self.v['hra'] = tk.StringVar()
        self.v['da'] = tk.StringVar()
        self.v['net_pay'] = tk.StringVar()
        frm = tk.LabelFrame(root, text='Details')
        frm.pack(fill='x', padx=8, pady=6)
        tk.Label(frm, text='Name').grid(row=0, column=0, padx=4, pady=4)
        tk.Entry(frm, textvariable=self.v['emp_name']).grid(row=0, column=1)
        tk.Label(frm, text='Designation').grid(row=0, column=2, padx=4, pady=4)
        tk.Entry(frm, textvariable=self.v['designation']).grid(row=0, column=3)
        tk.Label(frm, text='Basic').grid(row=1, column=0, padx=4, pady=4)
        tk.Entry(frm, textvariable=self.v['basic']).grid(row=1, column=1)
        tk.Label(frm, text='HRA').grid(row=1, column=2, padx=4, pady=4)
        tk.Entry(frm, textvariable=self.v['hra']).grid(row=1, column=3)
        tk.Label(frm, text='DA').grid(row=2, column=0, padx=4, pady=4)
        tk.Entry(frm, textvariable=self.v['da']).grid(row=2, column=1)
        tk.Label(frm, text='Net Pay').grid(row=2, column=2, padx=4, pady=4)
        tk.Entry(frm, textvariable=self.v['net_pay']).grid(row=2, column=3)
        bf = tk.Frame(root); bf.pack(pady=4)
        tk.Button(bf, text='Add', width=9, command=self.add).pack(side='left', padx=3)
```

```

tk.Button(bf, text='Update', width=9, command=self.update).pack(side='left', padx=3)
tk.Button(bf, text='Delete', width=9, command=self.delete).pack(side='left', padx=3)
tk.Button(bf, text='Clear', width=9, command=self.clear).pack(side='left', padx=3)
cols = ('id','emp_name','designation','basic','hra','da','net_pay')
self.tree = ttk.Treeview(root, columns=cols, show='headings', height=11)
for c in cols:
    self.tree.heading(c, text=c.title()); self.tree.column(c, width=100)
self.tree.pack(fill='both', expand=True, padx=8, pady=6)
self.tree.bind('<<TreeviewSelect>>', self.pick)
self.show()

def _vals(self):
    return [self.v[k].get() for k in ['emp_name', 'designation', 'basic', 'hra', 'da', 'net_pay']]
def add(self):
    c = db(); c.execute('INSERT INTO payroll(emp_name,designation,basic,hra,da,net_pay) VALUES(?, ?, ?, ?, ?, ?)',
                        self._vals())
    c.commit(); c.close(); self.clear(); self.show()

def update(self):
    if not self.sel: return
    c = db(); c.execute('UPDATE payroll SET emp_name=?,designation=?,basic=?,hra=?,da=?,net_pay=? WHERE id=?',
                        self._vals() + [self.sel])
    c.commit(); c.close(); self.clear(); self.show()

def delete(self):
    if not self.sel: return
    c = db(); c.execute('DELETE FROM payroll WHERE id=?', (self.sel,))
    c.commit(); c.close(); self.clear(); self.show()

def pick(self, e):
    r = self.tree.focus()
    if not r: return
    val = self.tree.item(r)['values']; self.sel = val[0]
    for i, k in enumerate(['emp_name', 'designation', 'basic', 'hra', 'da', 'net_pay']):
        self.v[k].set(val[i + 1])

def clear(self):
    self.sel = None
    for k in self.v: self.v[k].set('')

def show(self):
    self.tree.delete(*self.tree.get_children())
    c = db()
    for row in c.execute('SELECT * FROM payroll'):
        self.tree.insert('', 'end', values=row)
    c.close()

if __name__ == '__main__':
    root = tk.Tk(); Payroll(root); root.mainloop()

```

To auto-compute net pay before saving, replace the add () insert with:

```

def add(self):
    b = float(self.v['basic'].get() or 0)
    h = float(self.v['hra'].get() or 0)
    d = float(self.v['da'].get() or 0)
    self.v['net_pay'].set(b + h + d)
    c = db(); c.execute('INSERT INTO payroll(emp_name,designation,
        'basic,hra,da,net_pay) VALUES(?, ?, ?, ?, ?, ?)', self._vals())
    c.commit(); c.close(); self.clear(); self.show()

```

SAMPLE OUTPUT:

ID	EMP_NAME	DESIGNATION	BASIC	HRA	DA	NET_PAY
1	Ravi	AP	40000	8000	6000	54000.0

2	Meena	ASP	55000	11000	8250	74250.0
---	-------	-----	-------	-------	------	---------

RESULT:

Thus the GUI-based Employee Pay Roll System with full database features was developed successfully.

19. GUI-based Movie Ticket Reservation System

AIM:

To develop a GUI-based Movie Ticket Reservation System using Python (Tkinter) with a SQLite backend to book, view and cancel movie tickets, with full database features.

DATABASE DESIGN:

```
CREATE TABLE booking (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    customer     TEXT,  
    movie        TEXT,  
    show_time    TEXT,  
    seats        INTEGER,  
    amount       REAL);
```

SOURCE CODE (movie_booking.py):

```
import tkinter as tk  
from tkinter import ttk, messagebox  
import sqlite3  
  
def db():  
    c = sqlite3.connect('movie.db')  
    c.execute('''CREATE TABLE IF NOT EXISTS booking(  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        customer TEXT,  
        movie TEXT,  
        show_time TEXT,  
        seats INTEGER,  
        amount REAL)''')  
    return c  
  
class MovieBooking:  
    def __init__(self, root):  
        root.title('Movie Ticket Reservation'); root.geometry('660x480')  
        self.sel = None; self.v = {}  
        self.v['customer'] = tk.StringVar()  
        self.v['movie'] = tk.StringVar()  
        self.v['show_time'] = tk.StringVar()  
        self.v['seats'] = tk.StringVar()  
        self.v['amount'] = tk.StringVar()  
        frm = tk.LabelFrame(root, text='Details')  
        frm.pack(fill='x', padx=8, pady=6)  
        tk.Label(frm, text='Customer').grid(row=0, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['customer']).grid(row=0, column=1)  
        tk.Label(frm, text='Movie').grid(row=0, column=2, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['movie']).grid(row=0, column=3)  
        tk.Label(frm, text='Show').grid(row=1, column=0, padx=4, pady=4)  
        ttk.Combobox(frm, textvariable=self.v['show_time'], values=['10:00 AM', '01:30 PM', '04:00 PM', '07:30 PM', '10:00 PM']).grid(row=1, column=1)  
        tk.Label(frm, text='Seats').grid(row=1, column=2, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['seats']).grid(row=1, column=3)  
        tk.Label(frm, text='Amount').grid(row=2, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['amount']).grid(row=2, column=1)  
        bf = tk.Frame(root); bf.pack(pady=4)  
        tk.Button(bf, text='Add', width=9, command=self.add).pack(side='left', padx=3)  
        tk.Button(bf, text='Update', width=9, command=self.update).pack(side='left', padx=3)  
        tk.Button(bf, text='Delete', width=9, command=self.delete).pack(side='left', padx=3)  
        tk.Button(bf, text='Clear', width=9, command=self.clear).pack(side='left', padx=3)  
        cols = ('id', 'customer', 'movie', 'show_time', 'seats', 'amount')  
        self.tree = ttk.Treeview(root, columns=cols, show='headings', height=11)  
        for c in cols:  
            self.tree.heading(c, text=c.title()); self.tree.column(c, width=100)  
        self.tree.pack(fill='both', expand=True, padx=8, pady=6)
```

```

        self.tree.bind('<<TreeviewSelect>>', self.pick)
        self.show()

def _vals(self):
    return [self.v[k].get() for k in ['customer', 'movie', 'show_time', 'seats', 'amount']]
def add(self):
    c = db(); c.execute('INSERT INTO booking(customer,movie,show_time,seats,amount) VALUES
                        self._vals())
    c.commit(); c.close(); self.clear(); self.show()

def update(self):
    if not self.sel: return
    c = db(); c.execute('UPDATE booking SET customer=?,movie=?,show_time=?,seats=?,amount=
                        self._vals() + [self.sel])
    c.commit(); c.close(); self.clear(); self.show()

def delete(self):
    if not self.sel: return
    c = db(); c.execute('DELETE FROM booking WHERE id=?', (self.sel,))
    c.commit(); c.close(); self.clear(); self.show()

def pick(self, e):
    r = self.tree.focus()
    if not r: return
    val = self.tree.item(r)['values']; self.sel = val[0]
    for i, k in enumerate(['customer', 'movie', 'show_time', 'seats', 'amount']):
        self.v[k].set(val[i + 1])

def clear(self):
    self.sel = None
    for k in self.v: self.v[k].set('')

def show(self):
    self.tree.delete(*self.tree.get_children())
    c = db()
    for row in c.execute('SELECT * FROM booking'):
        self.tree.insert('', 'end', values=row)
    c.close()

if __name__ == '__main__':
    root = tk.Tk(); MovieBooking(root); root.mainloop()

```

SAMPLE OUTPUT:

ID	CUSTOMER	MOVIE	SHOW_TIME	SEATS	AMOUNT
1	Arjun	Vikram	06:00 PM	3	450.0
2	Priya	Leo	09:30 PM	2	300.0

Add books tickets; Delete cancels a booking.

RESULT:

Thus the GUI-based Movie Ticket Reservation System with full database features was developed successfully.

20. GUI-based Super Market Stock Maintenance System

AIM:

To develop a GUI-based Super Market Stock Maintenance System using Python (Tkinter) with a SQLite backend to add, update and monitor product stock, with full database features.

DATABASE DESIGN:

```
CREATE TABLE stock (  
    id            INTEGER PRIMARY KEY AUTOINCREMENT,  
    product       TEXT,  
    brand         TEXT,  
    stock_qty     INTEGER,  
    reorder       INTEGER,  
    price         REAL);
```

SOURCE CODE (supermarket_stock.py):

```
import tkinter as tk  
from tkinter import ttk, messagebox  
import sqlite3  
  
def db():  
    c = sqlite3.connect('stock.db')  
    c.execute('''CREATE TABLE IF NOT EXISTS stock(  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        product TEXT,  
        brand TEXT,  
        stock_qty INTEGER,  
        reorder INTEGER,  
        price REAL)''')  
    return c  
  
class SuperStock:  
    def __init__(self, root):  
        root.title('Super Market Stock'); root.geometry('660x480')  
        self.sel = None; self.v = {}  
        self.v['product'] = tk.StringVar()  
        self.v['brand'] = tk.StringVar()  
        self.v['stock_qty'] = tk.StringVar()  
        self.v['reorder'] = tk.StringVar()  
        self.v['price'] = tk.StringVar()  
        frm = tk.LabelFrame(root, text='Details')  
        frm.pack(fill='x', padx=8, pady=6)  
        tk.Label(frm, text='Product').grid(row=0, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['product']).grid(row=0, column=1)  
        tk.Label(frm, text='Brand').grid(row=0, column=2, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['brand']).grid(row=0, column=3)  
        tk.Label(frm, text='Stock').grid(row=1, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['stock_qty']).grid(row=1, column=1)  
        tk.Label(frm, text='Reorder').grid(row=1, column=2, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['reorder']).grid(row=1, column=3)  
        tk.Label(frm, text='Price').grid(row=2, column=0, padx=4, pady=4)  
        tk.Entry(frm, textvariable=self.v['price']).grid(row=2, column=1)  
        bf = tk.Frame(root); bf.pack(pady=4)  
        tk.Button(bf, text='Add', width=9, command=self.add).pack(side='left', padx=3)  
        tk.Button(bf, text='Update', width=9, command=self.update).pack(side='left', padx=3)  
        tk.Button(bf, text='Delete', width=9, command=self.delete).pack(side='left', padx=3)  
        tk.Button(bf, text='Clear', width=9, command=self.clear).pack(side='left', padx=3)  
        cols = ('id', 'product', 'brand', 'stock_qty', 'reorder', 'price')  
        self.tree = ttk.Treeview(root, columns=cols, show='headings', height=11)  
        for c in cols:  
            self.tree.heading(c, text=c.title()); self.tree.column(c, width=100)  
        self.tree.pack(fill='both', expand=True, padx=8, pady=6)
```

```

        self.tree.bind('<<TreeviewSelect>>', self.pick)
        self.show()

    def _vals(self):
        return [self.v[k].get() for k in ['product', 'brand', 'stock_qty', 'reorder', 'price']]
    def add(self):
        c = db(); c.execute('INSERT INTO stock(product,brand,stock_qty,reorder,price) VALUES(
                                self._vals())
        c.commit(); c.close(); self.clear(); self.show()

    def update(self):
        if not self.sel: return
        c = db(); c.execute('UPDATE stock SET product=?,brand=?,stock_qty=?,reorder=?,price=?
                                self._vals() + [self.sel])
        c.commit(); c.close(); self.clear(); self.show()

    def delete(self):
        if not self.sel: return
        c = db(); c.execute('DELETE FROM stock WHERE id=?', (self.sel,))
        c.commit(); c.close(); self.clear(); self.show()

    def pick(self, e):
        r = self.tree.focus()
        if not r: return
        val = self.tree.item(r)['values']; self.sel = val[0]
        for i, k in enumerate(['product', 'brand', 'stock_qty', 'reorder', 'price']):
            self.v[k].set(val[i + 1])

    def clear(self):
        self.sel = None
        for k in self.v: self.v[k].set('')

    def show(self):
        self.tree.delete(*self.tree.get_children())
        c = db()
        for row in c.execute('SELECT * FROM stock'):
            self.tree.insert('', 'end', values=row)
        c.close()

if __name__ == '__main__':
    root = tk.Tk(); SuperStock(root); root.mainloop()

```

A low-stock alert can be added by checking `stock_qty < reorder` in `show()` and tagging such rows, or in SQL: `SELECT * FROM stock WHERE stock_qty < reorder;`

SAMPLE OUTPUT:

ID	PRODUCT	BRAND	STOCK_QTY	REORDER	PRICE
1	Soap	Hamam	120	50	38.0
2	Shampoo	ClinicP	8	20	3.5

Row 2 is below reorder level -> flagged for restock.

RESULT:

Thus the GUI-based Super Market Stock Maintenance System with full database features was developed successfully.